

Documento del Proyecto - SGPR

> Documento completo del sistema de Gestión de Permisos y Reportes (SGPR). Desarrollado bajo metodología SCRUM como Responsable del desarrollo: Gabriela Urbinaa que ejecutó todas las actividades de análisis, diseño, desarrollo, pruebas e implementación.

Tabla de contenidos

- Visión (#visión)
- Misión (#misión)
- Objetivo general (#objetivo-general)
- Objetivos específicos (#objetivos-específicos)
- Propósito y justificación (#propósito-y-justificación)
- Alcance del proyecto (#alcance-del-proyecto)
- Descripción general del proyecto (#descripción-general-del-proyecto)
- Stakeholders (Interesados) (#stakeholders-interesados)
- Historias de usuario (Casos de uso) (#historias-de-usuario-casos-de-uso)
- Requerimientos funcionales (#requerimientos-funcionales)
- Requerimientos no funcionales (#requerimientos-no-funcionales)
- Backlog del Producto (#backlog-del-producto-ejemplos-de-épicas-y-user-stories)
- Sprint Backlog (#sprint-backlog-ejemplo-de-un-sprint-de-2-semanas)
- Acta de seguimiento (#acta-de-seguimiento-plantilla-con-entrada-real)
- Informe consolidado mensual (#informe-consolidado-mensual-de-acciones-de-implementación)
- Winframes y diseño de la interfaz (#winframes-y-diseño-de-la-interfaz)
- Diagrama de Base de Datos (MER) (#diagrama-de-base-de-datos-mer)
- Interfaces de entrada y salida del sistema (#interfaces-de-entrada-y-salida-del-sistema)
- Web Services básicos de los procesos (#web-services-básicos-de-los-procesos)
- Base de datos (backup) (#base-de-datos-backup)
- Desarrollo e implementación por módulos (#desarrollo-e-implementación-por-módulos)
- Formularios digitales (estructura) (#formularios-digitales-estructura)
- Módulos integrados y pruebas de instalación (#módulos-integrados-y-pruebas-de-instalación)
- Código y archivos fuentes (#código-y-archivos-fuentes)
- Informe de testeo / Pruebas realizadas (#informe-de-testeo--pruebas-realizadas)
- Manuales técnicos (#manuales-técnicos)
- Documentos de retrospectiva final del proyecto (#documentos-de-retrospectiva-final-del-proyecto)
- Anexos y recomendaciones finales (#anexos-y-recomendaciones-finales)

Visión

Ser la plataforma líder en la gestión y control de permisos y reposos laborales, ofreciendo informes precisos, exportes confiables (PDF/XLSX) y una experiencia intuitiva que facilite la toma de decisiones a nivel administrativo.

Misión

Proveer una solución web segura, auditable y eficiente para el registro, revisión y análisis de solicitudes laborales, automatizando cálculos de días laborables y generando reportes adaptables a las necesidades institucionales.

Objetivo general

Desarrollar e implementar una plataforma web que permita gestionar solicitudes de permisos y reposos, garantizando integridad de datos, trazabilidad y generación de reportes (visualización y exportes) para la administración.

Objetivos específicos

- Registrar y gestionar solicitudes de permisos y reposos por trabajador.
- Calcular automáticamente días continuos y días laborables considerando feriados.
- Permitir adjuntar justificantes y validar formatos permitidos.
- Generar reportes visuales y exportables (PDF/XLSX) que reproduzcan los gráficos de la vista.
- Registrar auditoría de acciones críticas con detalle cifrado.
- Mantener la seguridad y la confidencialidad de información sensible (cédulas, motivos) mediante cifrado en BD.

Propósito y justificación

El sistema centraliza la gestión de permisos, reduce errores administrativos, acelera procesos de revisión y aporta trazabilidad mediante auditorías. Justifica la inversión al disminuir tiempos administrativos y mejorar cumplimiento normativo.

Alcance del proyecto

Incluye: módulo de trabajadores, registro de solicitudes, gestión de estados (PENDIENTE/APROBADO/RECHAZADO), cálculo de días laborables, generación de reportes con gráficos, exportes PDF/XLSX, auditoría y control de accesos. No incluye: integración SSO externa (salvo Django auth), ni migraciones masivas automáticas.

Descripción general del proyecto

Aplicación Django con frontend que utiliza Chart.js para visualización. Exportes PDF se generan combinando captura client-side (canvas.toDataURL) y fallback server-side (matplotlib + ReportLab) si el canvas está "tainted". Se cifra información sensible con EncryptedTextField.

Stakeholders (Interesados)

- Administrador del sistema
- Recursos Humanos
- Trabajadores (usuarios finales)
- Equipo de IT/Soporte
- Auditoría interna

Historias de usuario (Casos de uso)

1. Como trabajador, quiero registrar una solicitud de permiso con motivo y adjunto para que quede registrada.
2. Como revisor, quiero ver las solicitudes pendientes y aprobar o rechazar con observaciones.
3. Como administrador, quiero generar un reporte PDF del trabajador que incluya gráficos y tablas.
4. Como auditor, quiero acceder al historial de acciones y ver detalles cifrados.
5. Como usuario, quiero exportar la vista de estadísticas a XLSX y PDF, y que los gráficos se reproduzcan fielmente.

Requerimientos funcionales

- RF1: Registrar Trabajador asociado a User de Django.
- RF2: Registrar Solicitud con tipo, fechas, motivo cifrado y adjunto.
- RF3: Validar extensiones de adjuntos (.pdf, .png, .jpg, .jpeg).
- RF4: Calcular diascontinuos y diaslaborables al guardar Solicitud.
- RF5: Visualizar estadísticas con gráficos interactivos (Chart.js).
- RF6: Exportar estadísticas a PDF/XLSX con gráficos incorporados.
- RF7: Registrar Auditoria en cada acción crítica.
- RF8: Control de acceso mediante permisos de Django.

Requerimientos no funcionales

- RNF1: Seguridad: cifrado de campos sensibles en repositorio.
- RNF2: Rendimiento: respuesta < 2s en consultas básicas para datasets pequeños.
- RNF3: Disponibilidad: despliegue reproducible en servidor (p.ej. Unicorn + Nginx) o mediante entorno virtual.
- RNF4: Portabilidad: compatible con PostgreSQL y SQLite para desarrollo.
- RNF5: Mantenibilidad: código documentado en español.

Backlog del Producto (Ejemplos de épicas y user stories)

- Épica: Gestión de Trabajadores
- US-101: CRUD Trabajador
- US-102: Vinculación con Django User
- Épica: Solicitudes
- US-201: Crear Solicitud con cálculo de días
- US-202: Adjuntar justificante y validación
- Épica: Reportes y Estadísticas
- US-301: Visualización Chart.js
- US-302: Exportar a PDF (cliente/servidor fallback)
- US-303: Exportar a XLSX

- Épica: Auditoría
- US-401: Registrar acción con detalles cifrados

Sprint Backlog (ejemplo de un sprint de 2 semanas)

- Tarea 1: Implementar modelos Trabajador y Solicitud (2 días)
- Tarea 2: Crear vista de listado y formulario (3 días)
- Tarea 3: Implementar Chart.js en estadísticas (2 días)
- Tarea 4: Implementar export PDF con fallback (4 días)
- Tarea 5: Pruebas unitarias y correcciones (3 días)

Acta de seguimiento (Plantilla con entrada real)

Sprint: 2026-05-01 → 2026-05-14

Asistentes: Desarrolladora (única persona)

Objetivos sprint: Implementar módulo de solicitudes y reportes básicos.

Avances: Modelos y vistas implementados; gráficos en UI funcionales; export PDF con capturas implementado.

Bloqueos: Canvas "tainted" en ciertos navegadores cuando se cargan recursos cross-origin.

Acciones: Implementar fallback en servidor (matplotlib); ajustar tamaños de imagen en PDF.

Informe consolidado mensual de acciones de implementación

- Mes: Mayo 2026
- Actividades:
 - Definición de requisitos y diseño ER
 - Implementación de modelos y lógica de negocio
 - Desarrollo de vistas y exportes (PDF/XLSX)
 - Pruebas manuales y correcciones
- Resultados: Plataforma en versión funcional para pruebas internas.

Winframes y diseño de la interfaz

- Estructura de páginas: Login, Lista Trabajadores, Formulario Solicitud, Panel de Estadísticas, Reportes.
- Diseño: HTML/CSS con Bootstrap básico; elementos Chart.js para gráficos.
- Nota: Wireframes y mockups se deben anexar como imágenes en carpeta `/docs/wireframes/`.

Diagrama de Base de Datos (MER)

Se incluye el archivo PlantUML con el MER en `erd.puml`.

> Nota: Para obtener una imagen (PNG/SVG) de `erd.puml`, usar la extensión PlantUML en VS Code o generar con plantuml.jar.

Interfaces de entrada y salida del sistema

- Entrada:
 - Formularios HTML para crear/editar Solicitud y Trabajador.
 - Archivos adjuntos (PDF/PNG/JPG).
 - Parámetros de filtrado en estadísticas (fechas, trabajador, tipo).
- Salida:
 - Vistas HTML con tablas y gráficos.
 - Exportes: PDF y XLSX descargables.
 - Registros de auditoría en BD.

Web Services básicos de los procesos

- `GET /api/solicitudes/` – Listar solicitudes (filtros opcionales)
- `POST /api/solicitudes/` – Crear solicitud (payload JSON + multipart adjunto)
- `GET /api/estadisticas/` – Obtener datos para gráficas (JSON)
- `POST /api/export/pdf/` – Generar PDF (acepta imágenes base64 o datos para fallback)

Base de datos (backup)

- Recomendación: backups diarios en PostgreSQL con `pgdump` en entorno de producción.
- Folder recomendado para copias: `/backups/` con rotación 7 días.

Desarrollo e implementación por módulos

- Módulo Trabajadores: modelos, vistas, templates, pruebas unitarias.
- Módulo Solicitudes: validaciones, cálculo de días, adjuntos.

- Módulo Estadísticas: recopilación de datos, generación de gráficos, exportes.
- Módulo Auditoría: registro de acciones con detalles cifrados.

Formularios digitales (estructura)

Ejemplo: Formulario de Solicitud

- trabajadorid (select)
- tipo (select)
- fechainicio (date)
- fechafin (date)
- motivo (textarea)
- adjunto (file)

Módulos integrados y pruebas de instalación

- La aplicación fue desplegada y probada localmente en entorno de desarrollo (Django runserver). Para producción, siga el Manual de Implantación para desplegar con Unicorn + Nginx o configure el servidor según las políticas de su organización.

Código y archivos fuentes

- Repositorio contiene código Django, templates, static, scripts de utilidades y archivos de configuración.
- Se entregan fuentes limpias, con comentarios en español y sin credenciales embebidas.

Informe de testeo / Pruebas realizadas

- Pruebas unitarias básicas sobre modelos y validaciones.
- Pruebas manuales de flujo: crear solicitud, aprobar/rechazar, generar reportes, exportes PDF/XLSX.
- Casos verificados: canvas válido, canvas tainted (fallback), adjuntos inválidos rechazados.

Manuales técnicos

- Manual de Usuario: guía para crear solicitudes, revisar y generar reportes (ubicado en ``/docs/manualusuario.md``).
- Manual de Implementación: pasos para desplegar en servidor (Unicorn + Nginx) y restaurar backups (``/docs/manualimplementacion.md``).

Manuales disponibles en /docs/:

- Manual de Usuario (`./docs/manualusuario.md`)
- Manual de Implantación (`./docs/manualimplementacion.md`)
- Manual para Mantenedores (`./docs/manualmantenedores.md`)
- Manual del Operador (`./docs/manualoperador.md`)
- Manual de Respaldo y Restauración (`./docs/manualrespaldorestauracion.md`)
- Manual de Pruebas / QA (`./docs/manualpruebas.md`)

Documentos de retrospectiva final del proyecto

- Resumen: trabajo desarrollado por única persona utilizando SCRUM adaptado (sprints cortos, reuniones diarias y revisiones). Lecciones aprendidas: importancia de definir paleta de colores compartida, manejar canvas tainted y limitar tamaño de payloads base64.

Anexos y recomendaciones finales

- Generar imagen ``erd.png`` desde ``erd.puml`` y colocar en ``/docs/`` para inclusión en la documentación.
- Crear carpeta ``/docs/wireframes/`` y añadir capturas de UI.
- Añadir scripts de backup y restauración en ``/ops/``.
- Documentar el proceso de generación de exports y las limitaciones por canvas tainted.

Metodología: SCRUM (Sprints, Backlog, Actas). Desarrolladora: una sola persona (full responsibility: análisis, diseño, desarrollo, pruebas, despliegue).

Fecha de creación del documento: 2026-06-03